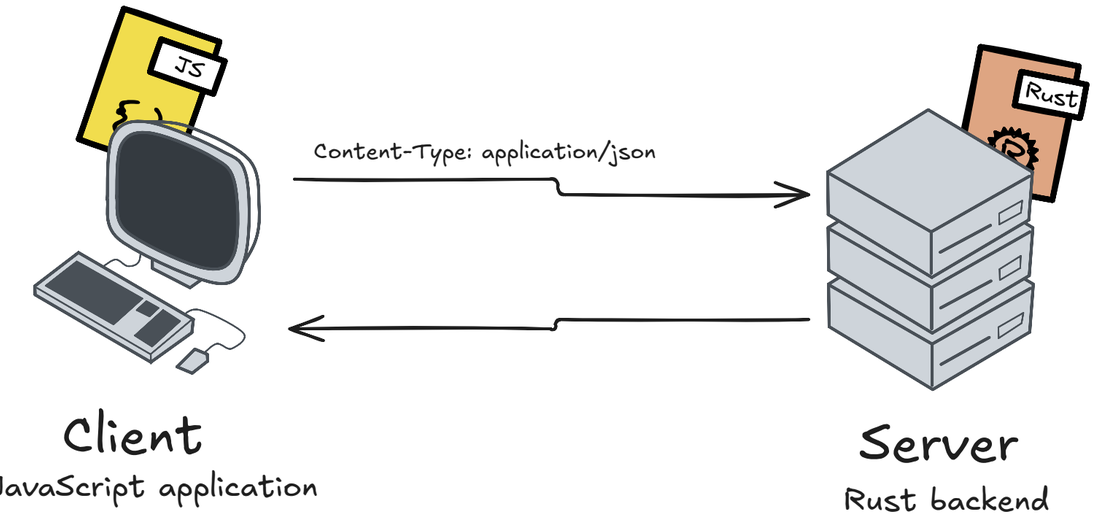
Serialization and Deserialization

How clients and servers make sense of the data being sent or received. ?



The problem is that each language represents data differently in memory.

JavaScript objects ≠ Rust structs

So they need a common format to exchange data

OSI Model
Common Standard

NOTE * Serialization and Deserialization happens at layer-6 of OSI model (presentation layer)

Presentation Layer: responsible for data translation, serialization, and encoding.

Serialization Standards

Text-Based

- JSON (JavaScript Object Notation) ✅
- XML
- YAML

Binary Formats (more efficient)

- Protocol Buffers
- MessagePack
- Avro

1. JavaScript Creates an Object

```
const user = {
name: "Ali",
age: 25
};
```

2. Serialization in JavaScript

```
JSON.stringify(user)
```

req ex

```
fetch("/user", {
method: "POST",
body: JSON.stringify(user)
});
```

↓

```
{"name":"Ali","age":25}
```

3. Data Reaches the Rust Server

```
{"name":"Ali","age":25}
```

But Rust does not automatically treat this as an object.

Rust needs to convert it into a struct. Like:

```
struct User {
name: String,
age: u32
}
```

4. Deserialization in Rust

The server deserializes the JSON into a Rust struct

```
let user: User = deserialize(json_data);
```

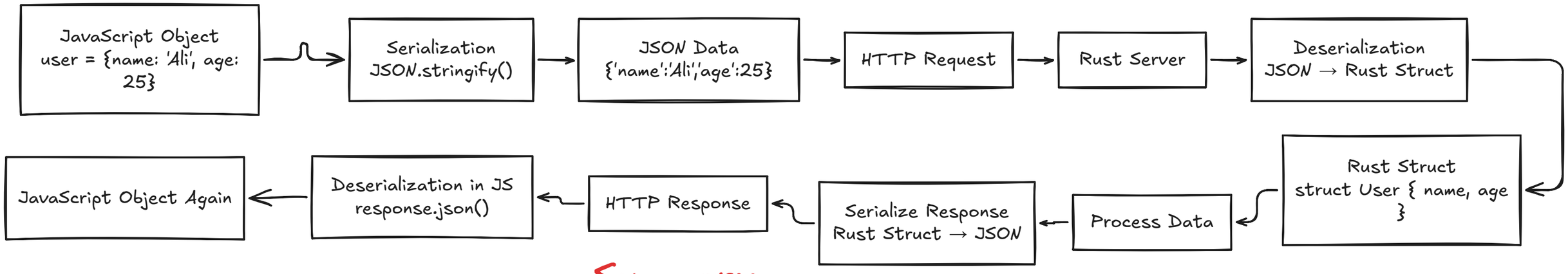
5. Server Sends a Response

If the server wants to send data back, it does the reverse.

Note That

Serialization → Object to transferable format

Deserialization → Transferable format to object



Summary

Serialization converts program objects into a transferable format (like JSON), and Deserialization reconstructs those objects from that format.